Machine Learning Optimization: Theory and Practice

Multiple Authors

May 16, 2025

Contents

1 Introduction to the course. Function classes for optimization. 1

1.1 The Optimization Problem . . . 1
1.2 First-Order Necessary Condition . . . 1
1.3 Second-Order Conditions . . . 1
1.3.1 Necessary Condition . . . 1
1.3.2 Sufficient Condition . . . 1
1.4 Taylor Expansion and Directional Derivatives . . . 2
1.5 Examples . . . 2
1.6 Convexity . . . 2
1.7 Concluding Remarks . . . 3

2 Gradient Descent. Convergence speed for different function classes. Examples. Finite-difference methods. 5

2.1 Introduction . . . 5
2.2 The Gradient Descent Algorithm . . . 5
2.3 Convergence Speed for Different Function Classes . . . 8
2.3.1 Convex Functions . . . 8
2.3.2 Strongly Convex Functions . . . 8
2.3.3 Non-convex Functions . . . 8
2.4 Variants of Gradient Descent . . . 8
2.4.1 Batch Gradient Descent . . . 8
2.4.2 Stochastic Gradient Descent . . . 8
2.4.3 Mini-batch Gradient Descent . . . 8
2.5 Finite-difference Methods . . . 8
2.5.1 Forward Difference . . . 8
2.5.2 Central Difference . . . 8
2.5.3 Higher-order Differences . . . 8
2.6 Applications in Machine Learning . . . 8
2.7 Conclusion . . . 8

3 Newton method. Invariance to linear scaling. Global and local convergence. Matrix decompositions for solving linear systems. Hessian correction schemes. 9

4 Conjugate Gradient method for solving linear systems, its convergence properties. Preconditioning. Non-linear CG. 11

5 Hessian-Free Newton (Newton-cg), its convergence speed. Convex sets. Introduction to Quasi-Newton methods. 13

6 Quasi-Newton methods SR1, BFGS and L-BFGS. Convex functions. 15

7 Constrained optimization problem, KKT theorem (first and second order necessary and sufficient conditions). Examples of analytical solving for constrained optimization, DeepFool adversarial attack. 17

8 Dual optimization problem, dual decomposition, duality for Wasserstein distance. Epigraph and reparameterization reformulations. Standard classes of convex constrained optimization problems. 19

9 Revised simplex method for Linear Programming. Log-Barrier method for constrained convex programming. 21

10 Non-smooth convex optimization: examples, subgradient method. Subdifferential calculus. 23

11 Proximal Gradient Method. Closed functions, Fenchel conjugate functions and conjugate (dual) norms 25

12 Projections and proximal operators. Differentiation through optimization methods. 27

13 Stochastic optimization. 29

Preface

This book covers various optimization methods used in machine learning, from fundamental algorithms to advanced techniques. Each chapter focuses on specific optimization methods and their applications in machine learning.

Chapter 1

Introduction to the course. Function classes for optimization.

Chapter 2

Introduction to Optimization Problems and Function Classes

Abstract:
This chapter introduces fundamental concepts in optimization, including necessary and sufficient conditions for local minima, directional derivatives, differential calculus in optimization, numerical precision considerations, and important function classes such as convex and Lipschitz continuous functions.

2.1 Introduction to Optimization Problems

The core optimization problem can be expressed as finding:

$$\min_{x \in \mathbb{R}^n} f(x) \tag{2.1}$$

Our objective is typically to find good local minima of the function $f : \mathbb{R}^n \to \mathbb{R}$.

2.2 Conditions for Local Minima

2.2.1 Necessary Conditions

For a function $f \in C^1$ with $x \in \text{int}(\text{dom} f)$, if x is a local minimum, then:

$$\nabla f(x) = 0 \tag{2.2}$$

This condition alone is not sufficient, as critical points where $\nabla f(x) = 0$ could be local maxima or saddle points. For example, in certain sparsity-oriented directions, we may have $\nabla^2 f(x)[d] < 0$.

2.2.2 Sufficient Conditions

For a function $f \in C^2$ with $x \in \text{int}(\text{dom} f)$, if:

$$\nabla f(x) = 0 \quad \text{and} \quad \nabla^2 f(x) \succ 0 \tag{2.3}$$

then x is a local minimum. Here, $\nabla^2 f(x) \succ 0$ means the Hessian matrix is positive definite.

2.3 Directional Derivatives and Optimality

For a vector $d \in \mathbb{R}^n$, the directional derivative of f at x in the direction d is defined as:

$$f'(x; d) := \lim_{\alpha \to 0^+} \frac{f(x + \alpha d) - f(x)}{\alpha} \tag{2.4}$$

If x is a local minimum, then for all directions d:

$$f'(x; d) \geq 0 \tag{2.5}$$

For differentiable functions, the directional derivative can be expressed as:

$$f'(x; d) = \nabla f(x)^T d \tag{2.6}$$

2.3.1 Example: Taylor Expansion

Consider $x \in [0, x_{\max})$ and the Taylor expansion of f:

$$f(x + \alpha d) = f(x) + \nabla f(x)^T (\alpha d) + \mathcal{O}(\alpha^2) \tag{2.7}$$

For $f'(0)$, we have:

$$f(x) = f(0) + f'(0)x + \mathcal{O}(x^2) \tag{2.8}$$

If $f(x) > f(0)$ for all $x > 0$, then $f'(0) \geq 0$.

2.4 Differential Calculus for Optimization

2.4.1 Definition of Differentiability

A function f is differentiable at $x \in \mathbb{R}^n$ if:

$$f(x + h) - f(x) = df(x)[h] + o(|h|) \tag{2.9}$$

where $df(x)[h]$ is a linear operator acting on h. For a function $f : \mathbb{R}^n \to \mathbb{R}$, the differential $df(x)[h]$ can be expressed as:

$$df(x)[h] = \nabla f(x)^T h \tag{2.10}$$

2.4.2 Second Differential

The second differential of f is defined as:

$$d^2 f(x)[h_1, h_2] = d(df(x)[h_1])(x)[h_2] \tag{2.11}$$

2.4.3 Example: Norm Squared Function

Consider the function $f(x) = \frac{1}{2}|x|^2$ for $x \in \mathbb{R}^n$. Then:

$$f(x) = \frac{1}{2}(x^T x) \quad df(x)[dx] \qquad = \frac{1}{2} \cdot 2x^T dx = x^T dx \quad \nabla f(x) = x \tag{2.12}$$

For the second differential:

$$d^2 f(x)[dx_1, dx_2] = d((x^T dx_1))(x)[dx_2] \qquad = (dx_1)^T dx_2 + x^T \cdot 0 \ = dx_1^T dx_2 \tag{2.13}$$

Thus, $\nabla^2 f(x) = I$, where I is the identity matrix.

2.5 Numerical Precision Considerations

In computational optimization, we must consider issues of numerical precision:

2.5.1 Machine Precision

Different numerical representations have varying precision levels:

- 64-bit: $\epsilon_m \approx 10^{-16}$
- 32-bit: $\epsilon_m \approx 10^{-7}$
- 16-bit: $\epsilon_m \approx 10^{-3}$

2.5.2 Impact on Directional Derivatives

When computing $f(x + \alpha d)$ for small α, numerical imprecision can dominate:

$$f(x + \alpha d) = f(x) + \alpha \nabla f(x)^T d + \mathcal{O}(\alpha^2) \tag{2.14}$$

For very small α, the term $\nabla f(x)^T d$ becomes indistinguishable from computational error.

2.5.3 Optimal Step Size

The optimal choice of α for numerical differentiation balances truncation error and rounding error:

$$\epsilon_{opt} \approx \sqrt{\epsilon_m} \tag{2.15}$$

This relationship can be visualized on a logarithmic plot of error versus step size.

2.6 Function Classes for Optimization

2.6.1 Convex Functions

A function f is convex if for all x, y in its domain and $\lambda \in [0, 1]$:

$$f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda) f(y) \tag{2.16}$$

For strictly convex functions, the inequality is strict for $x \neq y$ and $\lambda \in (0, 1)$.

First-Order Characterization

A differentiable function f is convex if and only if:

$$f(y) \geq f(x) + \nabla f(x)^T (y - x) \quad \forall x, y \in \mathrm{dom}(f) \tag{2.17}$$

2.6.2 Strongly Convex Functions

A function f is μ-strongly convex with $\mu > 0$ if:

$$f(y) \geq f(x) + \nabla f(x)^T (y - x) + \frac{\mu}{2}|y - x|^2 \quad \forall x, y \in \mathrm{dom}(f) \tag{2.18}$$

Note that if $f(x) - \frac{\mu}{2}|x|^2$ is convex, then f is μ-strongly convex.

2.6.3 Lipschitz Continuous Functions

A function $f \in C^1$ has a Lipschitz continuous gradient with constant $L > 0$ if:

$$|\nabla f(y) - \nabla f(x)| \leq L|y - x| \quad \forall x, y \in \mathrm{dom}(f) \tag{2.19}$$

Implications for Function Values

If f has an L-Lipschitz gradient, then:

$$f(y) \leq f(x) + \nabla f(x)^T(y - x) + \frac{L}{2}|y - x|^2 \quad \forall x, y \in \mathrm{dom}(f) \tag{2.20}$$

And also:

$$f(y) \geq f(x) + \nabla f(x)^T(y - x) - \frac{L}{2}|y - x|^2 \quad \forall x, y \in \mathrm{dom}(f) \tag{2.21}$$

2.7 Hessian Computation

For a function $f \in C^2$, the Hessian can be computed through the second differential:

$$d^2 f(x)[h] = h^T \nabla^2 f(x) h \tag{2.22}$$

This provides a way to verify second-order conditions for optimality.

2.8 Conclusion

This chapter has established fundamental concepts in optimization theory, including optimality conditions, directional derivatives, and important function classes. These concepts form the foundation for understanding and analyzing optimization algorithms for machine learning applications.

Chapter 3

Gradient Descent. Convergence speed for different function classes. Examples. Finite-difference methods.

Abstract:
This chapter explores gradient descent methods for optimization in machine learning. We analyze convergence rates for different function classes, provide practical examples, and discuss finite-difference approximations when gradients are not analytically available.

3.1 Introduction

Gradient descent is one of the most fundamental algorithms in optimization, serving as the cornerstone for many machine learning algorithms. This chapter introduces the method, its variants, convergence properties, and practical implementations.

3.2 The Gradient Descent Algorithm

Definition 3.1 (Gradient Descent)**.** Gradient descent is an iterative optimization algorithm that updates parameters in the direction of the negative gradient. Given a function $f : \mathbb{R}^n \to \mathbb{R}$, the update rule is:

$$x_{k+1} = x_k - \alpha_k \nabla f(x_k)$$

where $\alpha_k > 0$ is the step size or learning rate at iteration k.

Algorithm 3.1 Gradient Descent with Backtracking Line Search

Require: Function f, gradient ∇f, initial point x_0, tolerance $\epsilon > 0$, maximum iterations K
Require: Initial step size $\alpha_0 > 0$, backtracking parameters $0 < \beta < 1$, $0 < \sigma < 0.5$
Ensure: Approximate minimizer x^*

```
k ← 0
while ‖∇f(x_k)‖ > ε and k < K do
    d_k ← −∇f(x_k) {Descent direction}
    α ← α_0 {Reset step size}
    while f(x_k + αd_k) > f(x_k) + σα∇f(x_k)^T d_k do
        α ← βα {Reduce step size}
    end while
    x_{k+1} ← x_k + αd_k {Update point}
    k ← k + 1
end while
return x_k
```

Theorem 3.1 (Convergence of Gradient Descent for L-smooth Functions)**.** If f is L-smooth and convex, and gradient descent is run with constant step size $\alpha \leq \frac{1}{L}$, then after k iterations:

$$f(x_k) - f(x^*) \leq \frac{\|x_0 - x^*\|^2}{2\alpha k}$$

where x^* is a minimizer of f.

Proof: From L-smoothness, we have:

$$f(x_{k+1}) \leq f(x_k) + \nabla f(x_k)^T (x_{k+1} - x_k) + \frac{L}{2}\|x_{k+1} - x_k\|^2$$

Substituting $x_{k+1} = x_k - \alpha \nabla f(x_k)$:

$$f(x_{k+1}) \leq f(x_k) - \alpha\|\nabla f(x_k)\|^2 + \frac{L\alpha^2}{2}\|\nabla f(x_k)\|^2$$

When $\alpha \leq \frac{1}{L}$, we have $\frac{L\alpha^2}{2} \leq \frac{\alpha}{2}$, so:

$$f(x_{k+1}) \leq f(x_k) - \frac{\alpha}{2}\|\nabla f(x_k)\|^2$$

By convexity of f, we have $f(x_k) - f(x^*) \leq \nabla f(x_k)^T (x_k - x^*)$. Using the Cauchy-Schwarz inequality and some algebraic manipulations (omitted for brevity), we can establish the result. □

Claim 3.1 (Convergence for Strongly Convex Functions)**.** If f is μ-strongly convex and L-smooth, gradient descent with constant step size $\alpha = \frac{1}{L}$ converges linearly:

$$f(x_k) - f(x^*) \leq \left(1 - \frac{\mu}{L}\right)^k [f(x_0) - f(x^*)]$$

Example 3.1 (Quadratic Function)**.** Consider minimizing $f(x) = \frac{1}{2}x^T Ax + b^T x + c$ where A is positive definite. The gradient is $\nabla f(x) = Ax + b$, and gradient descent with optimal step size converges linearly with rate determined by the condition number of A.

Lemma 3.1 (Optimal Step Size)**.** For minimizing a quadratic function $f(x) = \frac{1}{2}x^T Ax + b^T x + c$, the optimal fixed step size is $\alpha = \frac{2}{\lambda_min(A)+\lambda_max(A)}$.

Example 3.2 (Python Implementation of Gradient Descent)**.** Below is a pseudocode implementation of gradient descent in Python-like syntax:

Algorithm 3.2 Gradient Descent Implementation

Require: f, ∇f, x_0, α, ϵ, max_iter
  $x \leftarrow x_0$
  $iter \leftarrow 0$
  $grad_norm \leftarrow \|\nabla f(x)\|$
  while $grad_norm > \epsilon$ and $iter < max_iter$ **do**
    $x \leftarrow x - \alpha \cdot \nabla f(x)$
    $grad_norm \leftarrow \|\nabla f(x)\|$
    $iter \leftarrow iter + 1$
    if $iter$ mod $100 = 0$ **then**
      Print "Iteration:", $iter$, "Function value:", $f(x)$, "Gradient norm:", $grad_norm$
    end if
  end while
  return x

For functions with expensive gradient computations, we can save computation by reusing the gradient:

Algorithm 3.3 Memory-Efficient Gradient Descent

Require: f, ∇f, x_0, α, ϵ, max_iter
  $x \leftarrow x_0$
  $iter \leftarrow 0$
  $grad \leftarrow \nabla f(x)$
  $grad_norm \leftarrow \|grad\|$
  while $grad_norm > \epsilon$ and $iter < max_iter$ **do**
    $x \leftarrow x - \alpha \cdot grad$
    $grad \leftarrow \nabla f(x)$
    $grad_norm \leftarrow \|grad\|$
    $iter \leftarrow iter + 1$
  end while
  return x

3.3 Convergence Speed for Different Function Classes

3.3.1 Convex Functions

3.3.2 Strongly Convex Functions

3.3.3 Non-convex Functions

3.4 Variants of Gradient Descent

3.4.1 Batch Gradient Descent

3.4.2 Stochastic Gradient Descent

3.4.3 Mini-batch Gradient Descent

3.5 Finite-difference Methods

3.5.1 Forward Difference

3.5.2 Central Difference

3.5.3 Higher-order Differences

3.6 Applications in Machine Learning

3.7 Conclusion

Chapter 4

Newton method. Invariance to linear scaling. Global and local convergence. Matrix decompositions for solving linear systems. Hessian correction schemes.

Chapter 5

Conjugate Gradient method for solving linear systems, its convergence properties. Preconditioning. Non-linear CG.

Chapter 6

Hessian-Free Newton (Newton-cg), its convergence speed. Convex sets. Introduction to Quasi-Newton methods.

Chapter 7

Quasi-Newton methods SR1, BFGS and L-BFGS. Convex functions.

Chapter 8

Constrained optimization problem, KKT theorem (first and second order necessary and sufficient conditions). Examples of analytical solving for constrained optimization, DeepFool adversarial attack.

Chapter 9

Dual optimization problem, dual decomposition, duality for Wasserstein distance. Epigraph and reparameterization reformulations. Standard classes of convex constrained optimization problems.

Chapter 10

Revised simplex method for Linear Programming. Log-Barrier method for constrained convex programming.

Chapter 11

Non-smooth convex optimization: examples, subgradient method. Subdifferential calculus.

Chapter 12

Proximal Gradient Method. Closed functions, Fenchel conjugate functions and conjugate (dual) norms

Chapter 13

Projections and proximal operators. Differentiation through optimization methods.

Chapter 14

Stochastic optimization.